Systems Requirements Specification: Privacy Preserving Data Analytics Engine (E-Privacy)

Dilmith D. L. S., Hansana O. G. V., Vidyananda K. H. K.

August 21, 2026

1 Introduction

1.1 Purpose

This document is a concrete requirements specification for a system that computes on encrypted data without ever decrypting it server-side.

1.2 Scope

The Engine allows a Data Owner (e.g. a hospital, bank or research institute) to outsource statistical aggregation of sensitive records to an untrusted cloud environment while cryptographically guaranteeing that the cloud never observes plaintext values. The scope covers functional and non-functional requirements, system architecture, and data flow; selection and empirical justification of the underlying cryptographic techniques is addressed separately in the cryptographic design specification.

1.3 Assumptions

These are the assumptions that were made to scope this SRS:

A1. The Engine targets **additive aggregate analytics** (sum, average, count, weighted average) as the primary workload, consistent with the Feasibility PoC. Complex joins, non-linear functions (e.g. median, standard deviation without a secondary protocol) are deferred to a later phase.

A2. The **Data Owner is a single trusted entity** per deployment (e.g. one hospital) that holds the private key; multi-owner federated aggregation is a future extension requiring threshold cryptography.

A3. The cloud provider is treated as **honest-but-curious**: it will follow the protocol correctly but will attempt to learn as much as possible from ciphertext and metadata. Active/Byzantine cloud behavior is mitigated only by the verifiable computation log, not prevented outright.

A4. Network transport (TLS 1.3) and at-rest storage encryption (AES-256) are assumed already compliant per the organization's existing infrastructure and are treated as satisfied preconditions.

A5. Private key material is managed and stored securely on the Data Owner side, following the organization's existing key-management practices; the Engine does not implement its own key-custody mechanism but integrates with whatever secure key store the Data Owner already operates.

2 System Architecture

The Engine partitions responsibility across a trusted Data Owner domain and an untrusted Cloud Analytics Engine domain, directly closing the "Data-in-Use" gap.

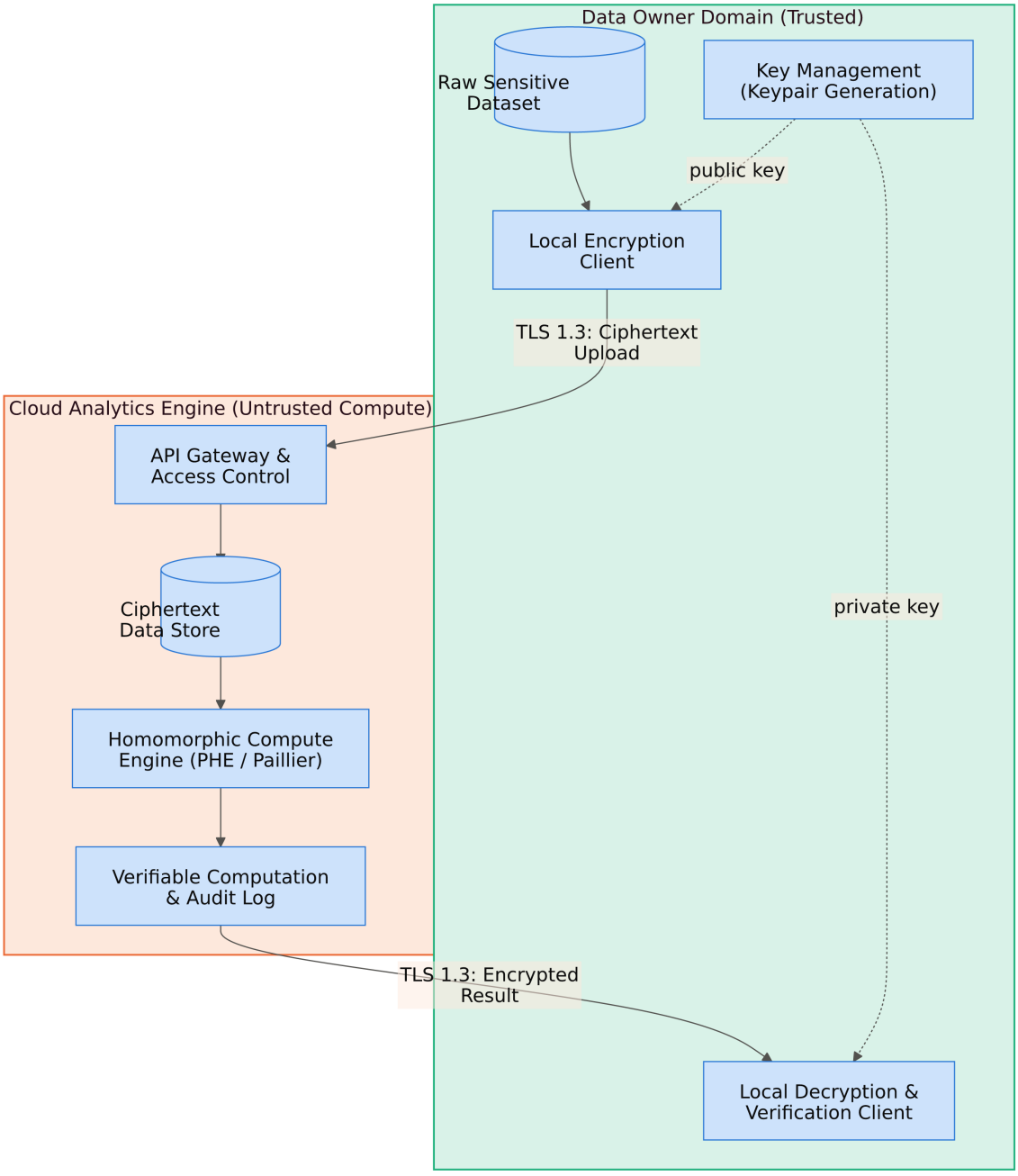


Figure 1: High-level system architecture: encryption and decryption occur exclusively within the trusted Data Owner domain; the cloud only ever manipulates ciphertext.

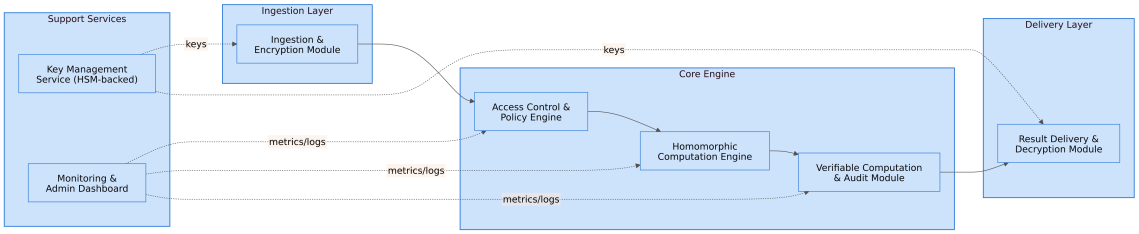


Figure 2: Component/module breakdown of the Engine across the ingestion, core computation, support, and delivery layers.

2.1 End-to-End Data Flow

Figure 3 traces a single analytics request end-to-end, showing exactly where the private key is used (only within the Data Owner domain, steps 1–2 and 9–10) and where the cloud only ever handles ciphertext.

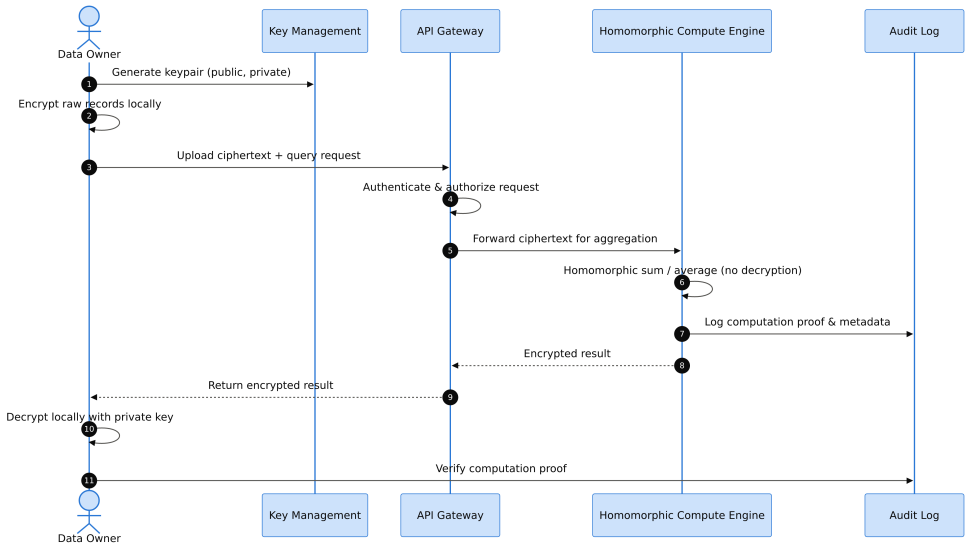


Figure 3: Sequence of operations for a homomorphic aggregation query.

3 Functional Requirements

ID	Requirement	Description
FR-1	Local Key Generation	The Data Owner client must generate a Paillier (or equivalent additively homomorphic) keypair locally; the private key must never leave the trusted domain.
FR-2	Client-Side Encryption	Raw records must be encrypted on the Data Owner's infrastructure before any network transmission.
FR-3	Homomorphic Aggregation	The cloud engine must compute sum, average, and count directly over ciphertext without decryption, using additive homomorphism and plaintext-scalar multiplication.
FR-4	Access Control & Policy Engine	The API Gateway must authenticate and authorize every query request against a role-based policy before ciphertext is released to the compute engine.
FR-5	Result Return & Local Decryption	Aggregated encrypted results must be returned to the Data Owner and decrypted only within the trusted domain.
FR-6	Verifiable Computation Log	Every computation must emit an auditable, tamper-evident log entry (operation type, input ciphertext hashes, timestamp) so the Data Owner can verify correctness.
FR-7	Multi-Key-Length Support	The Engine must support configurable key bit-lengths (at minimum 1024/2048/3072) so operators can trade off security margin against latency.
FR-8	Admin Monitoring Dashboard	Operators must be able to observe query throughput, latency, and storage consumption without access to any private key or plaintext.

Table 1: Functional Requirements

3.1 Requirement Prioritization

Figure 4 maps each functional requirement against implementation effort and business impact, informing the phased delivery plan: homomorphic aggregation and local key management are the two "must-ship-first" capabilities, since every other requirement depends on them.

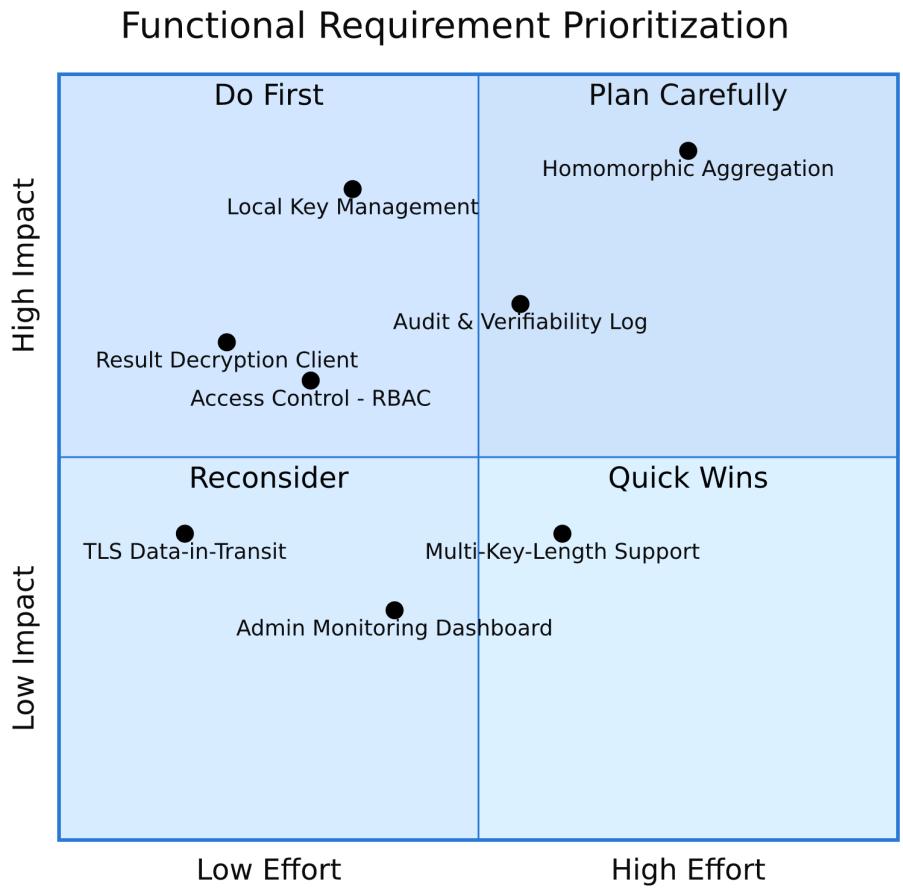


Figure 4: Functional requirement prioritization by impact vs. implementation effort.

4 Non-Functional Requirements

Category	Requirement
Confidentiality	No plaintext sensitive value may exist outside the Data Owner's trusted domain at any point in the pipeline.
Integrity & Verifiability	Every cloud-side computation must be independently verifiable against the audit log; unauthorized modification of stored ciphertext must be detectable (e.g. via authenticated encryption / MAC on ciphertext blobs).
Performance	Analytics latency must remain within an operator-configured SLA per key length; a 2048-bit PHE deployment should target sub-5-minute aggregation latency for a 2M-record batch (see the cryptographic design specification for the supporting benchmark).
Scalability	The compute engine must horizontally scale ciphertext summation across worker nodes, since Paillier summation is embarrassingly parallel.
Storage Efficiency	Ciphertext storage expansion must be bounded and predictable; PHE at 2048-bit keys is budgeted at $\leq$ 1.1 GB per 100 MB of plaintext (an $\sim$11x expansion), versus FHE's $\sim$650x (see the cryptographic design specification).
Availability	The API Gateway and compute engine must meet a 99.5% uptime target, independent of the Data Owner's local infrastructure availability.
Auditability	All access-control decisions and computations must be logged with sufficient metadata for post-hoc compliance review (e.g. HIPAA, GDPR-style audits), without logging plaintext.
Usability	The client-side encryption/decryption library must expose a single-call API (encrypt/aggregate/decrypt) so integration effort for a new Data Owner is under one engineer-week.

Table 2: Non-Functional Requirements

5 Requirements Traceability

Security Requirement	Addressed By	Req. ID
Confidentiality	Client-side encryption; no server-side decryption ever occurs	FR-1, FR-2, FR-3
Integrity	Tamper-evident audit log; authenticated ciphertext storage	FR-6
Anonymity	Aggregation-only query surface prevents record-level linkage; direct identifiers never leave the trusted domain	FR-2, FR-4
Verifiability	Verifiable computation log independently checkable by the Data Owner	FR-6

Table 3: Traceability from security requirements to functional requirements.

6 Constraints and Future Work

- Paillier's lack of native ciphertext-ciphertext multiplication limits the Engine to linear aggregate statistics in this phase; variance/standard-deviation queries require either a two-round protocol or a partial FHE fallback.
- Encryption latency at higher key lengths motivates a streaming/incremental ingestion pipeline rather than a batch-encrypt-then-query model, to be detailed in the subsequent design deliverable.
- Multi-owner federated queries and differential-privacy noise injection for output perturbation are explicitly deferred (A2, A1) and should be scoped in a follow-on SRS revision.

7 Conclusion

This SRS specifies a concrete system design for privacy-preserving analytics. By confining plaintext exclusively to the Data Owner's trusted domain and performing all cloud-side analytics via Partially Homomorphic Encryption, the Engine closes the Data-in-Use vulnerability window while remaining within practical storage and latency budgets. The empirical benchmarking that justifies the specific cryptosystem and key-length selection is presented in the accompanying cryptographic design specification.